Deep Learning Milestone 2: Training, Experiments, Results, and Contributions

Group 7 Pedro Fanica (54346) Aarthi Perumpillichira (66404) Maja Skwaroń (66083)

1. Problem Statement

This project investigates how a Graph Neural Network (GNN)-based music knowledge graph, integrating user interaction history, musical features (such as pitch statistics and tempo), and song metadata (including artists and genres), can be used to predict user-specific music preferences. While many music recommendation algorithms already exist and are widely used, most of them rely primarily on user interaction data and similarity measures between users. Incorporating musical features into the recommendation process may provide a richer and more structured representation of songs, enabling the model to capture deeper relationships between musical content and user preferences. By leveraging a knowledge graph combined with GNNs, this project aims to model these complex, multi-relational dependencies and explore their potential for more accurate and context-aware music preference prediction.

2. Background and Context

In order to ensure a suitable choice of models and their architecture, reasonable evaluation metrics, as well as the choice of relevant musical features and appropriate graph design, we review the prior studies on music recommendation systems, collaborative filtering techniques, and graph-based learning methods, including Graph Neural Networks (GNNs). Such as research performed by [1] which is a knowledge graph-enhanced recommendation framework that jointly learns from user-item interactions and graph-based representations, and represents a state-of-the-art approach for improving recommendation accuracy and addressing challenges such as data sparsity and cold start.

3. Data

For this project, 2 major data sources were used:

- **Lakh MIDI Dataset (LMD)** - A large-scale collection of MIDI files (machine-readable representation of musical structure) representing music in symbolic form (notes, timing, instruments).
- **Million Song Dataset (MSD)** - A dataset developed by The Echo Nest, containing audio-derived features such as tempo, key, or loudness metadata for songs, as well as a user taste profile subset, which is used to define our recommendation target.

4. Methodology

A high-level overview of the project methodology is shown in 1

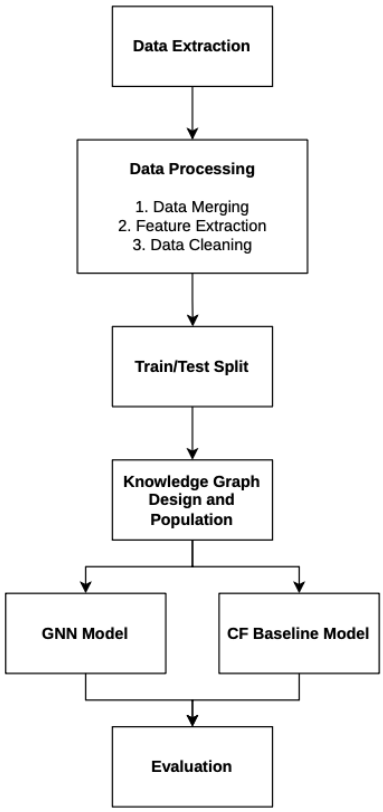

Figure 1. End-to-end pipeline overview. Raw MIDI and MSD data are aligned and cleaned; symbolic features are extracted and compressed by the autoencoder; the enriched KG is constructed and loaded into GraphDB; KG embeddings initialise node features; the HGT is trained on the PyG heterogeneous graph; and recommenders are evaluated at multiple K values.

4.1. Data pipeline

The first step of the data processing methodology involved the extraction and linking of the Lakh MIDI and Million Song Dataset (MSD). The alignment between MIDI files and audio tracks was based on precomputed similarity scores obtained using Dynamic Time Warping (DTW) by Raffel (2016). In this process, both MIDI and audio signals were first converted into chroma-based representations over time, and then compared to determine their similarity. The threshold for acceptable similarity score of 0.55 was set during the filtering step, in order to ensure reliable alignment. The remaining files and duplicates were omitted resulting in a dataset of 18383 tracks. The next step in the data preparation process concerned processing and subsequently linking user interaction data extracted from the Echo Nest Taste Profile dataset, consisting of unique song and user identifiers as well as a play count for each recorded pairing. The user data was filtered to retain only the songs that also exist in the previously matched Lakh/MSD dataset. Additionally, instances with fewer than 5 remaining song interactions were removed, to address the cold start problem (the inclusion of users with not enough listening history to train and evaluate recommendations for them). After the filtering step the dataset consists of 286,889 unique users, and 7,050 unique songs.

After the data integration and filtering process, the subsequent step includes feature extraction, engineering, and selection. In our data, the attributes of each song can be either of metadata or symbolic music feature type. The first ones are those derived directly from audio analysis and metadata features. Some of these features are related to song's artist, such as their name, genre, location, familiarity, or hotness. Other features are related to the songs themselves, including song release year, the name of an album it is a part of, the song hotness, duration, or even algorithmic estimation of danceability or energy score calculated from listener point of view. The symbolic music features are those constructed based on various representations (MIDI, audio, musicXML) of the LAKH midi files using tools such as jSymbolic, music21, and include features such as tempo, pitch range, note density, rhythm features, pitch histograms, or melodic features. A large volume of features can be extracted from each track using these libraries, some of which are less useful and/or unsuitable for integration into a knowledge graph. Therefore some of these features are excluded/removed based on these and other traits.

The final step of the data pipeline consists of designing and populating the knowledge graph. The graph schema is defined using a structured ontology, where core entities such as users (listener class), musical tracks, artists, genres, and certain musical features are represented as nodes, and with the relationships between these entities modeled as edges. For example, user-song interactions are captured through relations such as listened, while songs are linked to artists (performed by), genres (has genre), and symbolic or audio-derived features (has music feature). Additionally, hierarchical relationships between concepts, such as subclasses of musical features (e.g., rhythm, dynamics, pitch statistics), are incorporated to enrich the semantic structure of the graph. The graph is populated by integrating the filtered user interaction data with the Lakh/MSD dataset and the extracted musical features, using shared identifiers such as song_id and MIDI file mappings. This unified representation enables the combination of relational, contextual, and musical information, forming the foundation for subsequent graph-based learning and recommendation models. Figure 2 shows the ontology of the knowledge graph linking users, songs, and musical features that are be used to train the GNN for recommendation prediction.

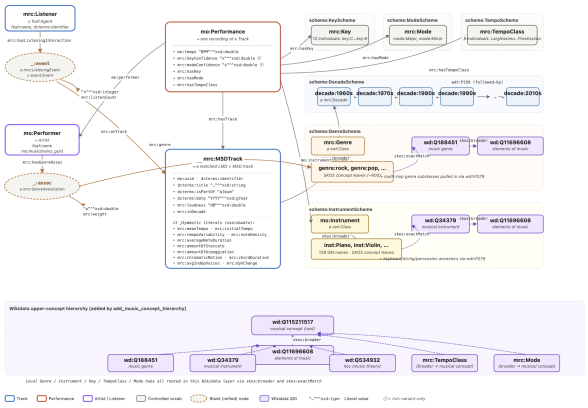

Figure 2. An overview of the ontology used to

4.2. Baselines

Three baseline models were chosen to establish a baseline for comparison against the KG + GNN pipeline, these were:

- **MostPopular** ranks all unseen items by global play frequency identically for every user.
- **KNN-CF**, a user–user collaborative filter purely based on user taste data and compensating for song popularity.
- **XGBoost Hybrid**, a gradient-boosted LTR model which uses a 300-dimensional feature vector per (user, track) pair that concatenates the 128-dim user-profile and track AE embeddings with 44 categorical features: one-hot key, mode, and tempo class; target-encoded genre and artist; and a multi-hot instrument indicator. This design isolates the GNN contribution by fusing content and behaviour without any graph structure.

4.3. Heterogeneous Graph Transformer (HGT)

The Heterogeneous Graph Transformer (HGT) [2] learns separate transformation and attention parameters for each (source-type, edge-type, target-type) triple, preserving the graph's typed structure throughout message passing. This

makes HGT particularly well suited to our multi-relational music KG, where node types span users, tracks, artists, genres, instruments, keys, modes, tempo classes, and decades. An additional advantage over opaque embedding models is that HGT's attention weights can be traced back to specific edge types and neighbouring entities, providing recommendation rationales that are directly grounded in KG structure — an important property for a domain where users benefit from understanding why a track was suggested. For these reasons, the HGT was selected as the ML model to be used for the KG + GNN pipeline.

5. Training Methods

Autoencoder. A fully connected autoencoder compresses the 1,525-dimensional jSymbolic [3]/music21 [4] feature vector of each track to a 128-dimensional bottleneck via encoder layers $1526 \to 512 \to 256 \to 128$ (symmetric decoder), ReLU + dropout 0.2. Trained with Adam ($lr{=}10^{-3}$, batch 256) to minimise MSE for up to 200 epochs; early stopping triggered if $\Delta{<}10^{-4}$ over 15 epochs.

KGE (RotatE [5] / ComplEx [6]). Both models are trained for 100 epochs on 807,955 triples (listening edges excluded). Adam optimiser, entity_dim=128, batch 32,768, 256 negatives per positive. Output: 256-dimensional vectors for all 304,110 entities.

HGT [2]. 2 message-passing layers, 4 attention heads, hidden dim 128, dropout 0.1. Track nodes: 384-dim input (128-dim AE $\|$ 256-dim RotatE); other types: 256-dim KGE. Per-type MLP head $\to$ 64-dim ℓ_2-normalised output. Trained 200 epochs, Adam ($lr{=}10^{-3}$, wd 10^{-4}) with cosine-annealing. Checkpoint selected by composite Overall Score at $K{=}10$ on validation.

Loss: logit-adjusted listwise cross-entropy with soft engagement targets $\tilde{p}_{ui} \propto \log(1 + c_{ui})$:

$$s_i = \frac{\hat{\mathbf{u}}^\top \hat{\mathbf{t}}_i}{\tau} + \lambda \log p_i, \quad \mathcal{L} = -\frac{1}{|B|} \sum_{u,i} \tilde{p}_{ui} \log \mathrm{softmax}(\mathbf{s}_u)_i$$

where $\tau{=}0.1$ and p_i is the Laplace-smoothed marginal listen probability.

KNN-CF. User–user cosine similarity on the ℓ_2-normalised interaction matrix; $k{=}6$ neighbours selected by validation sweep.

XGBoost Hybrid. Top-200 candidates per user retrieved via AE-profile cosine similarity, re-ranked with a gradient-boosted LTR model (`rank:ndcg@10`, 300 rounds, early stopping 30) on 300-dim feature vectors.

6. Experiments

- **Autoencoder compression**: transductive 200-epoch run over 7,013 tracks; MSE monitored per epoch.
- **KGE model comparison**: RotatE vs. ComplEx as HGT node initialisers.
- **Init ablation**: full 200-epoch HGT with RotatE-based vs. Gaussian random node features.
- **Logit-adjustment ablation**: 30-epoch sweeps over $\lambda \in \{0.0, 0.05, 0.1, 0.2, 0.5\}$.
- **Multi-K evaluation**: all models at $K \in \{5, 10, 20, 50, 100\}$; Wilcoxon + Friedman tests on a shared 5,000-user subsample.

Evaluation protocol. A stratified 70/10/20 train/val/test split by genre and decade preserves popularity and content distributions across folds; song features and KG structure are visible to all splits. Per-user metrics (Precision, Recall, HR, MRR, NDCG) are supplemented by two catalogue-level diversity metrics: coverage $\mathrm{Cov}{=}|\bigcup_u R_u|/|\mathcal{I}|$ and popularity bias $\mathrm{PB}@K$. These are fused into a composite score

$$\mathrm{Overall}@K = 0.6\,\mathrm{NDCG}@K + 0.2\,\mathrm{Cov} + 0.2(1 - \mathrm{PB}@K),$$

which also serves as the HGT checkpoint criterion. Evaluation user sample similarity determination uses paired Wilcoxon signed-rank and Friedman omnibus tests on a shared 5,000-user subsample to ensure that the users included in the evaluation sample are comparable.

7. Results and Learning Curves

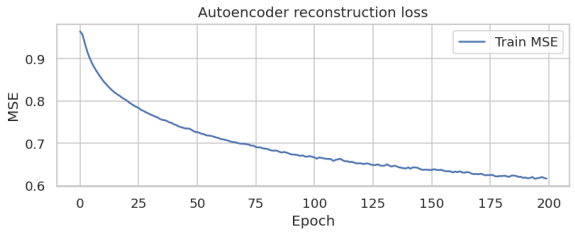

Figure 3. Autoencoder MSE (200 epochs): 0.964 $\to$ 0.616. No early stopping triggered; steepest descent in first 75 epochs. RotatE KGE final loss: 0.356; ComplEx: 0.381 (100 epochs each).

KNN-CF achieves the best Overall Score (0.421), dominated by its 91.6% catalogue coverage. MostPopular leads on per-user NDCG but concentrates recommendations on 0.6% of the catalogue. XGBoost reaches 66.7% coverage but suffers from a retrieval bottleneck that limits NDCG@10 to 0.044. Random init marginally outperforms RotatE init for HGT (ΔRecall $+0.007$, ΔNDCG $+0.005$), suggesting KGE features are subsumed by HGT's own relational learning and/or that the methodology for learning the embeddings

Table 1. Test benchmark at $K=10$ (all 284,864 test users). Overall $= 0.6 \cdot \text{NDCG} + 0.2 \cdot \text{Cov} + 0.2(1 - \text{PB})$.

Model	Rec.	NDCG	HR	MRR	Cov.	PB	Ovrl.
MostPopular	**.129**	**.114**	**.200**	**.090**	.006	.611	.148
KNN-CF	.103	.106	.169	.087	**.916**	.130	**.421**
XGBoost	.038	.044	.072	.035	.667	**.118**	.336
HGT†	.001	.001	.002	.000	.046	.006	.209
HGT (self)*	.111	.071	—	—	.399	.070	—

† Directed-eval mismatch (trained undirected, eval directed).

* Consistent self-eval (undirected inference, same as training).

Table 2. HGT ablations at $K=10$. Init: RotatE vs. random (200 epochs). λ sweep (30 epochs).

	Config	Recall	NDCG	Cov	PB
Init	RotatE	.111	.071	.399	.070
	Random	.118	.076	.407	.069
λ	0.00	.125	.081	.006	.321
	0.05	.124	.079	.006	.319
	0.10	.118	.073	.007	.305
	0.20	.110	.068	.015	.280
	0.50	.077	.048	.088	.123

is flawed. The debiasing term λ trades ranking quality for coverage: $\lambda=0.5$ triples coverage (0.6% $\rightarrow$ 8.8%) while halving Recall (0.125 $\rightarrow$ 0.077). All pairwise Wilcoxon and t-test comparisons are significant ($p < 0.001$).

HGT evaluation note. The HGT benchmark row reflects a directed-evaluation mismatch: the model was trained with undirected message passing but evaluated under directed edges, preventing it from reaching user nodes during inference. Under consistent undirected inference (HGT self row) it recovers Recall@10 0.111 and Coverage 39.9%. Retraining with directed edges is the direct remediation [2].

HGT explainability. Unlike purely collaborative methods, HGT attention weights can be inspected per edge type to identify which entity types — shared genres, co-instrumented tracks, shared tempo class — most influenced a recommendation. This provides human-interpretable explanations grounded in KG structure without additional modelling overhead, a qualitative advantage not captured by the quantitative metrics above. Future work should extend baselines to graph-collaborative methods such as LightGCN [7] or KGCN [8] for a fuller comparison.

8. Discussion

The results expose a clear ranking-vs-diversity trade-off. MostPopular maximises NDCG by exploiting long-tail play-count structure, yet ignores 99.4% of the catalogue; KNN-CF trades a modest NDCG drop (0.106 vs. 0.114) for near-complete coverage (91.6%), tripling the Overall Score (0.421 vs. 0.148). This confirms that popularity exploitation is not viable when diversity is part of the objective. This also touches upon a flaw/limitation in evaluation methodology: Popularity is likely a factor in whether or not a user has already listened to a song, but not necessarily a factor in whether or not a user will enjoy a song. As any static dataset can only include music a user has listened to, a recommendation being absent from the users profile does not inherently mean the user will not enjoy the song. However, any method for compensating for this would require closer integration with the user themselves.

The XGBoost hybrid reveals a retrieval bottleneck: constraining candidates to 200 items caps reachable NDCG regardless of LTR quality. The low NDCG@10 (0.044) is a first-stage ceiling, not a re-ranking failure; expanding the candidate pool via approximate nearest-neighbour search would be the highest-value fix.

The HGT ablations yield two non-obvious findings. First, RotatE initialisation offers no benefit over random: HGT's own message passing re-learns structural features within a few epochs, making the 100-epoch KGE pre-training step redundant. Second, the logit-adjustment λ acts as a smooth ranking-vs-diversity knob ($\lambda=0$ maximises Recall; $\lambda=0.5$ boosts coverage $8\times$ at half the Recall), and checkpoint selection by Overall Score automatically lands at a balanced point. Finally, the directed/undirected evaluation mismatch is the single most consequential artefact: HGT's apparent collapse in Table 1 is entirely an evaluation protocol issue; under consistent self-eval it is competitive with KNN-CF (Recall@10 0.111, Coverage 39.9%). See Appendix for all learning curves.

9. Member Contributions

Pedro Fanica (54346) Pipeline architecture, `nb_env.py` bootstrap, GraphDB upload and SPARQL validation, HGT training loop with logit-adjusted loss, latent-space GMM/UMAP analysis (§13), cold-start persona GUI (§14).

Aarthi Perumpillichira (66404) KG design and construction: OWL ontology schema, `kg_builder.py`, Wikidata entity alignment, decade temporal chain, rich KG reification, SPARQL query library and GraphDB inspection.

Maja Skwaroń (66083) Data processing pipeline: LakhMSDLinker, user taste profile filtering, jSymbolic and music21 feature extraction, symbolic-feature autoencoder, KNN-CF and XGBoost baselines, evaluation protocol and statistical significance tests.

References

[1] X. Liu, Z. Yang, and J. Cheng. Music recommendation algorithms based on knowledge graph and multi-task feature learning. *Scientific Reports*, 14:2055, 2024. doi: 10.1038/s41598-024-52463-z. URL https://doi.org/10.1038/s41598-024-52463-z. 1

[2] Ziniu Hu, Yuxiao Dong, Kuansan Wang, and Yizhou Sun. Heterogeneous graph transformer. In *Proceedings of The Web Conference 2020*, WWW '20, pages 2704–2710, 2020. doi: 10.1145/3366423.3380027. 2, 3, 4

[3] Cory McKay, Julie Cumming, and Ichiro Fujinaga. Jsymbolic 2.2: Extracting features from symbolic music for use in musicological and mir research. In *Proceedings of the 19th International Society for Music Information Retrieval Conference (ISMIR)*, pages 348–354, 2018. 3

[4] Hogue Cuthbert, Ariza and Oberholzer. *music21*. music21, 3rd edition, May 2026. Available at https://music21.org/music21docs/. 3

[5] Zhiqing Sun, Zhi-Hong Deng, Jian-Yun Nie, and Jian Tang. RotatE: Knowledge graph embedding by relational rotation in complex space, 2019. Accepted at ICLR 2019. 3

[6] Théo Trouillon, Johannes Welbl, Sebastian Riedel, Éric Gaussier, and Guillaume Bouchard. Complex embeddings for simple link prediction, 2016. Accepted at ICML 2016. 3

[7] Xiangnan He, Kuan Deng, Xiang Wang, Yan Li, Yongdong Zhang, and Meng Wang. LightGCN: Simplifying and powering graph convolution network for recommendation. In *Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 639–648, 2020. doi: 10.1145/3397271.3401063. 4

[8] Hongwei Wang, Miao Zhao, Xing Xie, Wenjie Li, and Minyi Guo. Knowledge graph convolutional networks for recommender systems. In *Proceedings of the World Wide Web Conference (WWW)*, pages 3307–3313, 2019. doi: 10.1145/3308558.3313417. 4

Appendix: Learning Curves

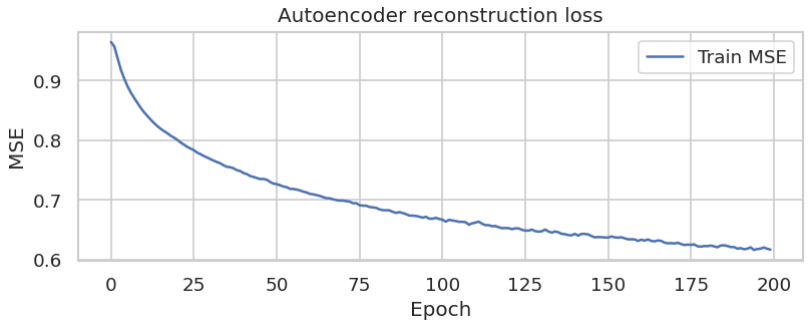

Figure 4. Autoencoder training loss (MSE) over 200 epochs. Loss falls from 0.964 to 0.616; the steepest descent occurs in the first 75 epochs. Early stopping ($\Delta < 10^{-4}$ over 15 epochs) never triggered, indicating continued — albeit diminishing — compression improvement throughout training.

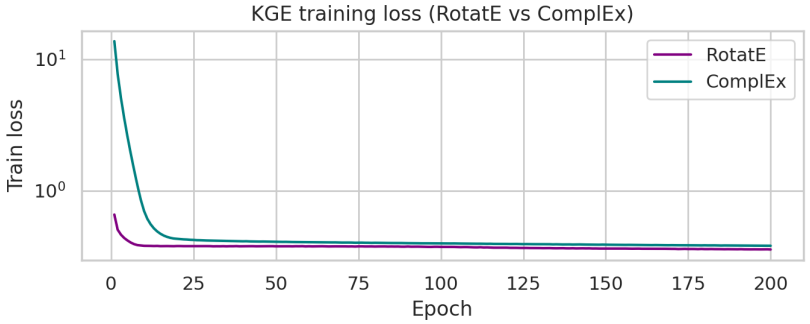

Figure 5. KGE training loss (log scale) for RotatE and ComplEx over 200 epochs. The two models operate on different absolute scales (ComplEx $\approx 13 \to 3$; RotatE $\approx 0.7 \to 0.4$), so a logarithmic y-axis keeps both curves legible. RotatE converges to a lower absolute loss (final: 0.356 vs. ComplEx: 0.381), but this difference does not translate into downstream HGT ranking improvements (Table 2).

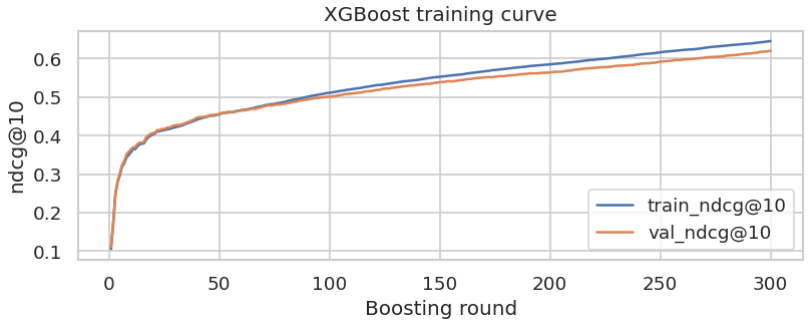

Figure 6. XGBoost LTR training curve (`rank:ndcg@10` objective, up to 300 rounds, early stopping at 30). The model converges rapidly and the early-stopping criterion terminates training well before 300 rounds, confirming the 300-candidate retrieval bottleneck as the limiting factor rather than under-training.

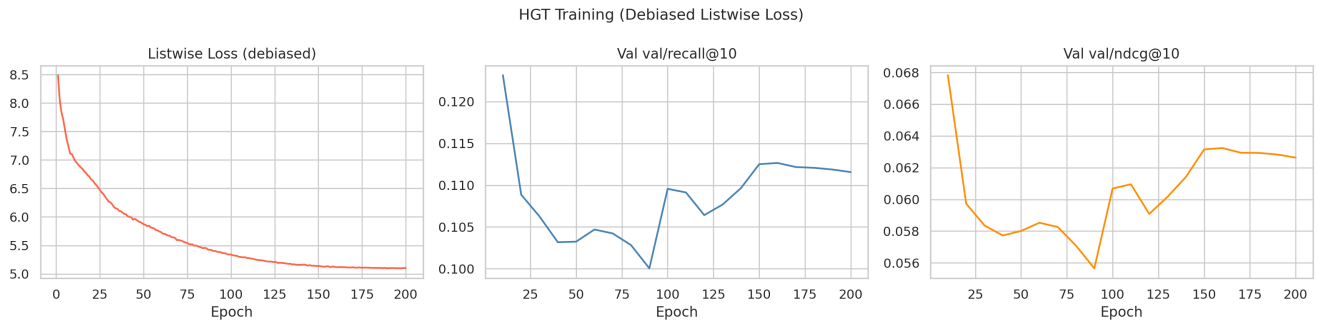

Figure 7. HGT training curves (200 epochs, RotatE initialisation). *Left*: listwise debiased loss decreases monotonically. *Centre*: validation Recall@10 improves steadily and plateaus. *Right*: validation NDCG@10 mirrors the Recall trend. All three panels confirm stable optimisation with no signs of overfitting or divergence; the final checkpoint is selected by the composite Overall Score on the validation fold.